PostalMS

Unit Testing Report

MSTest Framework | 78 Tests | 0 Failures | 100% Pass Rate

1. Test Summary

All 78 unit tests pass successfully with 0 failures and a 100% pass rate. Tests are organised into 9 classes covering all three custom data structures, parcel price calculation, tracking ID validation, Gmail validation, stamp price calculation with VAT and city location detection.

Test Class	Tests	Passed	Failed	Coverage Area
HashTableTests	12	**12**	0	Tests the Custom Hash Table implementation using separate chaining for collision resolution
BSTTests	13	**13**	0	Tests the Custom Binary Search Tree implementation
QueueTests	10	**10**	0	Tests the Custom Queue implementation using a singly linked list
PriceCalculationTests	9	**9**	0	Tests the parcel price calculation formula used in ParcelsView
TrackingIDTests	6	**6**	0	Tests the tracking ID generation and validation logic
DataStructureIntegrationTests	5	**5**	0	Tests all three custom data structures working together as they do in the real PostalMS application
GmailValidationTests	8	**8**	0	Tests the Gmail validation logic enforced in LoginForm, RegisterForm, StampsView and ParcelsView
StampPriceTests	6	**6**	0	Tests the stamp price calculation in StampsView
CityLocationTests	8	**8**	0	Tests the city detection logic in FindUsView used to show PostalMS locations near the user based on their registered city
TOTAL	**78**	**78**	**0**	**All modules covered**

2. Test Environment

Item	Details
Testing Framework	MSTest (Microsoft.VisualStudio.TestTools.UnitTesting)
Language	C# .NET Framework 4.7.2
IDE	Visual Studio 2022
Test Runner	Visual Studio Test Explorer
Total Tests	78 tests across 9 test classes
Duration	Under 600ms for all 78 tests
Pass Rate	100% -- 78 passed, 0 failed
Test Architecture	Self-contained -- local copies of data structures, no project reference needed
Gmail Validation Coverage	LoginForm, RegisterForm, StampsView and ParcelsView all covered
Stamp Price Coverage	Standard delivery, Express delivery, Click and Collect (free), VAT and service fee
City Coverage	All 7 cities: London, Manchester, Birmingham, Leeds, Bristol, Edinburgh, Glasgow

3. Testing Approach

Each test follows the Arrange-Act-Assert pattern. Test classes are completely self-contained with local copies of all three data structures inside the test project, meaning they build and run without any project reference or database connection.

Exception tests use try-catch blocks instead of Assert.ThrowsException to ensure compatibility with all versions of MSTest. The three new Sprint 4 test classes (GmailValidationTests, StampPriceTests and CityLocationTests) mirror the exact logic from the production code files and run in under 10ms combined.

4. Detailed Test Results

HashTableTests (12 tests)

Tests the Custom Hash Table implementation using separate chaining for collision resolution. Covers add, search, delete, count, clear, resize and ContainsKey operations. Time complexity O(1) average.

Test Name	Input / Action	Expected Result	Outcome
Add_ThenSearch_ReturnsCorrectValue	Add PS-2026-00001 with Pending then search	Returns Pending status	**PASS**
Search_KeyNotInTable_ReturnsNull	Search for a key never added	Returns null	**PASS**
Add_SameKeyTwice_UpdatesValueNotCount	Add same key twice with	Count stays 1, value updated	**PASS**

Test Name	Input / Action	Expected Result	Outcome
	different statuses		
Add_FiveParcels_AllRetrievable	Add 5 different tracking IDs	All 5 retrievable by key	**PASS**
Count_EmptyTable_IsZero	Check count on new empty table	Count equals 0	**PASS**
Count_AfterAddingItems_IncreasesCorrectly	Add items and check count	Count increases correctly	**PASS**
IsEmpty_NewTable_ReturnsTrue	Check IsEmpty on new table	Returns true	**PASS**
IsEmpty_AfterAddingItem_ReturnsFalse	Add one item then check IsEmpty	Returns false	**PASS**
Delete_ExistingKey_ReturnsTrueAndRemovesItem	Delete existing tracking ID	Returns true, item removed, count 0	**PASS**
Delete_NonExistentKey_ReturnsFalse	Delete a key never added	Returns false	**PASS**
ContainsKey_ExistingKey_ReturnsTrue	Check ContainsKey for existing key	Returns true	**PASS**
Add_50Items_TriggersResizeAndAllStillRetrievable	Add 50 items to trigger resize	All 50 items still accessible after resize	**PASS**

BSTTests (13 tests)

Tests the Custom Binary Search Tree implementation. Covers insert, search, delete, in-order traversal, FindMin, Contains and Clear. Time complexity O(log n) average.

Test Name	Input / Action	Expected Result	Outcome
Insert_ThenSearch_ReturnsCorrectValue	Insert PS-2026-00003 then search	Returns correct status	**PASS**
Search_NonExistentKey_ReturnsNull	Search for key not in tree	Returns null	**PASS**
Insert_DuplicateKey_UpdatesValueKeepsCountAt1	Insert same key twice	Value updated, count stays 1	**PASS**

Test Name	Input / Action	Expected Result	Outcome
InOrder_FiveItemsInsertedOutOfOrder_ReturnsSortedResult	Insert 5 IDs out of order	InOrder returns all 5 sorted ascending	**PASS**
InOrder_EmptyTree_ReturnsEmptyList	Call InOrder on empty tree	Returns empty list with count 0	**PASS**
FindMin_ThreeNodes_ReturnsSmallestKey	Insert 3 nodes then FindMin	Returns node with smallest key	**PASS**
FindMin_EmptyTree_ReturnsNull	Call FindMin on empty tree	Returns null	**PASS**
Delete_ExistingNode_ReturnsTrueAndRemoves	Delete existing node	Returns true, node removed, count decreases	**PASS**
Delete_NonExistentKey_ReturnsFalse	Delete key not in tree	Returns false	**PASS**
Count_AfterInserts_IsCorrect	Insert nodes and check count	Count matches number inserted	**PASS**
Clear_EmptiesTreeAndResetsCount	Insert items then Clear	Count 0, IsEmpty true, nodes gone	**PASS**
Contains_ExistingKey_ReturnsTrue	Check Contains for inserted key	Returns true	**PASS**
Contains_MissingKey_ReturnsFalse	Check Contains for missing key	Returns false	**PASS**

QueueTests (10 tests)

Tests the Custom Queue implementation using a singly linked list. Covers enqueue, dequeue, peek, FIFO ordering, IsEmpty, Count, ToList and Clear. Time complexity O(1) for all core operations.

Test Name	Input / Action	Expected Result	Outcome
Enqueue_ThenDequeue_ReturnsSameValue	Enqueue DEL-001 then dequeue	Returns DEL-001	**PASS**
Queue_ThreeItems_MaintainsFIFOOrder	Enqueue DEL-001, 002, 003 then dequeue all	Returned in FIFO order: DEL-001 first	**PASS**
Dequeue_EmptyQueue_ThrowsInvalidOperationException	Dequeue from empty queue	InvalidOperationException thrown via try-catch	**PASS**
Peek_ReturnsFrontItemWithoutRemoving	Enqueue 2 items then Peek	Returns front item, count unchanged at 2	**PASS**
Peek_EmptyQueue_ThrowsInvalidOperationException	Peek at empty queue	InvalidOperationException thrown via try-catch	**PASS**
IsEmpty_NewQueue_ReturnsTrue	Check IsEmpty on new queue	Returns true	**PASS**
IsEmpty_AfterEnqueue_ReturnsFalse	Enqueue one item then check IsEmpty	Returns false	**PASS**
IsEmpty_AfterEnqueueAndFullDequeue_ReturnsTrue	Enqueue then dequeue all	Returns true after all dequeued	**PASS**
Count_IncreasesOnEnqueueDecreasesOnDequeue	Track count through enqueue and dequeue	Count correctly tracks additions and removals	**PASS**
Clear_EmptiesQueueAndResetsCount	Enqueue items then Clear	Count 0, IsEmpty true	**PASS**

PriceCalculationTests (9 tests)

Tests the parcel price calculation formula used in ParcelsView. Formula: (ServicePrice + Weight x 1.2) x SizeMultiplier x InternationalMultiplier. Covers all service types, sizes and international zones.

Test Name	Input / Action	Expected Result	Outcome
Price_SmallGroundZeroWeight_ReturnsBasePrice	0kg Small Ground domestic	Returns 2.99 base service price	**PASS**
Price_IncreasesWithWeight	Compare 0kg vs 5kg same service	5kg price greater than 0kg price	**PASS**
Price_LargeParcelMoreExpensiveThanSmall	Compare Small x1.0 vs Large x2.5	Large more expensive than Small	**PASS**
Price_ExpressMoreExpensiveThanGround	Compare Ground 2.99 vs Express 9.99	Express more expensive than Ground	**PASS**
Price_ServicesInCorrectCostOrder	Compare Ground, Priority, Express, Next Day	Ground < Priority < Express < Next Day	**PASS**
Price_InternationalMoreExpensiveThanDomestic	Compare domestic x1.0 vs international x1.8	International more expensive	**PASS**
Price_Zone4MoreExpensiveThanZone1	Compare Zone 1 x1.8 vs Zone 4 x3.8	Zone 4 more expensive than Zone 1	**PASS**
Price_IsAlwaysPositive	Minimum parcel price	Price always greater than 0	**PASS**
Price_IsRoundedToTwoDecimalPlaces	Check decimal places of result	Price rounded to maximum 2 decimal places	**PASS**

TrackingIDTests (6 tests)

Tests the tracking ID generation and validation logic. Format: PS-YYYY-NNNNN where YYYY is the current year and NNNNN is a zero-padded 5 digit number.

Test Name	Input / Action	Expected Result	Outcome
GenerateTrackingID_FollowsCorrectFormat	Generate tracking ID for parcel 1	Format matches PS-YYYY-NNNNN and IsValid returns true	**PASS**

Test Name	Input / Action	Expected Result	Outcome
GenerateTrackingID_ParcelNumberZeroPaddedToFiveDigits	Generate ID for parcel 1	Number part is 00001, exactly 5 characters	**PASS**
GenerateTrackingID_DifferentNumbers_ProduceDifferentIDs	Generate IDs for parcels 1, 2 and 100	All three IDs are unique	**PASS**
GenerateTrackingID_ContainsCurrentYear	Generate tracking ID	Year part matches current year	**PASS**
IsValidTrackingID_ValidFormats_ReturnTrue	Validate PS-2026-00001, PS-2026-99999	Both return true	**PASS**
IsValidTrackingID_InvalidFormats_ReturnFalse	Validate empty, null, wrong prefix, short year	All invalid formats return false	**PASS**

DataStructureIntegrationTests (5 tests)

Tests all three custom data structures working together as they do in the real PostalMS application. Simulates add parcel, update status, delete parcel, sorted display and delivery queue processing.

Test Name	Input / Action	Expected Result	Outcome
SimulateAddParcel_AddsToHashTableAndBSTSimultaneously	Add parcel to Hash Table and BST	Both structures contain parcel with correct status	**PASS**
SimulateUpdateStatus_BothStructuresReflectNewStatus	Update status in both structures and enqueue delivery	Both structures show updated status, Queue has 1 delivery	**PASS**
SimulateDeleteParcel_RemovedFromBothStructures	Delete parcel from	Neither structure	**PASS**

Test Name	Input / Action	Expected Result	Outcome
	Hash Table and BST	contains the parcel	
BSTInOrder_ReturnsSameParcelsAsHashTable_ButSorted	Insert 5 parcels out of order into both structures	BST InOrder sorted, count matches Hash Table	**PASS**
DeliveryQueue_ProcessedInFIFOOrder	Enqueue 3 deliveries then dequeue all	Processed in order: DEL-001, DEL-002, DEL-003	**PASS**

GmailValidationTests (8 tests)

Tests the Gmail validation logic enforced in LoginForm, RegisterForm, StampsView and ParcelsView. All emails must end with @gmail.com as required by professor feedback. Non-Gmail emails are rejected before any database call.

Test Name	Input / Action	Expected Result	Outcome
ValidGmail_ValidEmail_ReturnsTrue	Validate john@gmail.com	Returns true	**PASS**
ValidGmail_WrongDomain_ReturnsFalse	Validate john@hotmail.com	Returns false -- wrong domain	**PASS**
ValidGmail_MissingUsername_ReturnsFalse	Validate @gmail.com with no username	Returns false -- no username before @	**PASS**
ValidGmail_EmptyString_ReturnsFalse	Validate empty string	Returns false	**PASS**
ValidGmail_YahooEmail_ReturnsFalse	Validate test@yahoo.com	Returns false -- not Gmail domain	**PASS**
ValidGmail_OutlookEmail_ReturnsFalse	Validate user@outlook.com	Returns false -- not Gmail domain	**PASS**
ValidGmail_ValidEmailWithNumbers_ReturnsTrue	Validate john123@gmail.com	Returns true	**PASS**
ValidGmail_ValidEmailWithDot_ReturnsTrue	Validate john.smith@gmail.com	Returns true	**PASS**

StampPriceTests (6 tests)

Tests the stamp price calculation in StampsView. Formula: (Stamps + Delivery) + 20% VAT + 2% Service Fee. Covers First Class, Second Class, Click and Collect free delivery, Express delivery and bulk orders.

Test Name	Input / Action	Expected Result	Outcome
StampPrice_FirstClass_SingleStamp_CorrectTotal	1 x First Class GBP 1.10 + GBP 1.50 delivery	Total 3.17 (subtotal 2.60 + VAT 0.52 + service 0.05)	**PASS**
StampPrice_SecondClass_TenStamps_CorrectTotal	10 x Second Class GBP 0.75 + GBP 1.50 delivery	Total 10.98 (subtotal 9.00 + VAT 1.80 + service 0.18)	**PASS**
StampPrice_FreeCollection_NoDeliveryFee	Compare standard delivery vs Click and Collect	Click and Collect cheaper due to zero delivery fee	**PASS**
StampPrice_ExpressDelivery_HigherThanStandard	Compare standard GBP 1.50 vs express GBP 3.99	Express total higher than standard total	**PASS**
StampPrice_LargerQuantity_HigherTotal	Compare 1 stamp vs 25 stamps same type	25 stamps total higher than 1 stamp total	**PASS**
StampPrice_VATAlwaysApplied	Calculate total and compare to subtotal	Total always greater than subtotal due to 20% VAT	**PASS**

CityLocationTests (8 tests)

Tests the city detection logic in FindUsView used to show PostalMS locations near the user based on their registered city. Covers all 7 supported UK cities including case-insensitive matching and unknown city fallback to London.

Test Name	Input / Action	Expected Result	Outcome
DetectCity_LondonAddress_ReturnsLondon	Address contains London	Returns London	**PASS**
DetectCity_ManchesterAddress_ReturnsManchester	Address contains Manchester	Returns Manchester	**PASS**

Test Name	Input / Action	Expected Result	Outcome
DetectCity_UnknownCity_DefaultsToLondon	Address contains unknown city not in supported list	Defaults to London	**PASS**
DetectCity_BirminghamAddress_ReturnsBirmingham	Address contains Birmingham	Returns Birmingham	**PASS**
DetectCity_CaseInsensitive_ReturnsCorrectCity	Address contains LEEDS in uppercase	Returns Leeds correctly	**PASS**
SupportedCities_ContainsAllRequiredCities	Check all 7 cities in supported list	All 7 present: London, Manchester, Birmingham, Leeds, Bristol, Edinburgh, Glasgow	**PASS**
SupportedCities_HasAtLeastSevenCities	Check count of supported cities list	Count is 7 or more	**PASS**
DetectCity_GlasgowAddress_ReturnsGlasgow	Address contains Glasgow	Returns Glasgow	**PASS**

5. Conclusion

All 78 unit tests pass with 0 failures and a 100% pass rate. The tests confirm that all three custom data structures (Hash Table, BST and Queue) operate correctly, the parcel price formula produces accurate results, tracking IDs are generated in the correct format, Gmail validation correctly enforces @gmail.com only in all four forms (LoginForm, RegisterForm, StampsView and ParcelsView), stamp price calculation correctly applies 20% VAT and 2% service fee, and city detection correctly identifies all 7 supported UK cities from user addresses.

The test file is fully self-contained with no external project reference needed, ensuring reliable builds across all development environments.